

Tenant isolation — recommended approach

How we make sure one institution can never see another's data: a layered design, the senior developer's view-layer idea folded in, and how it deploys.

In one paragraph

Keep **one database with shared tables** (every row tagged with its tenant). Protect it with **five independent locks**, so a single mistake can't leak data: the tenant is checked on every request, set safely on each transaction, the app uses a **restricted database login, row-level security** filters every table and fails closed, and tests stop an unprotected table shipping. On top, a **per-tenant layer of views** is the only door for reporting and data exports. Large or sensitive institutions get their **own install**; the same code serves both.

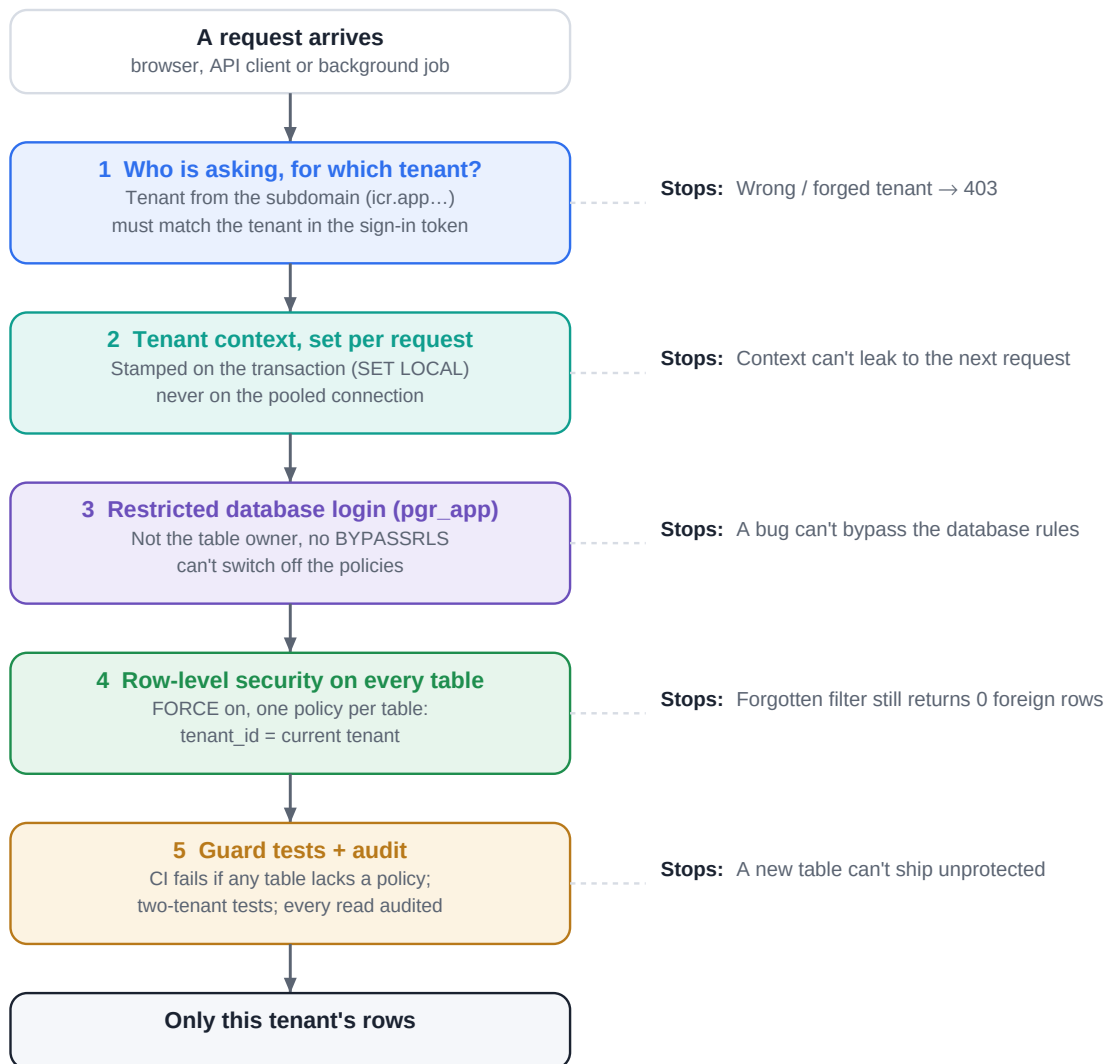
1. Where cross-tenant leaks actually happen

A design is only as safe as its weakest path in. These are the paths we have to close:

Leak path	Example	Closed by
A query forgets the tenant filter	A new screen lists “all students”	Lock 4 (row-level security)
Tenant context bleeds between requests	A pooled connection keeps the last tenant	Lock 2 (per transaction)
The app login can switch the rules off	Owner role bypasses policies	Lock 3 (restricted login)
Someone asks for another tenant	Edits the URL or token	Lock 1 (tenant must match)
A new table ships without protection	Added in a later feature	Lock 5 (guard tests)
Reporting / BI connects to raw tables	A warehouse reads everything	Tenant view layer (section 3)
Files, exports, caches, AI context	Shared folder or cache key	Per-tenant prefixes and keys (section 5)

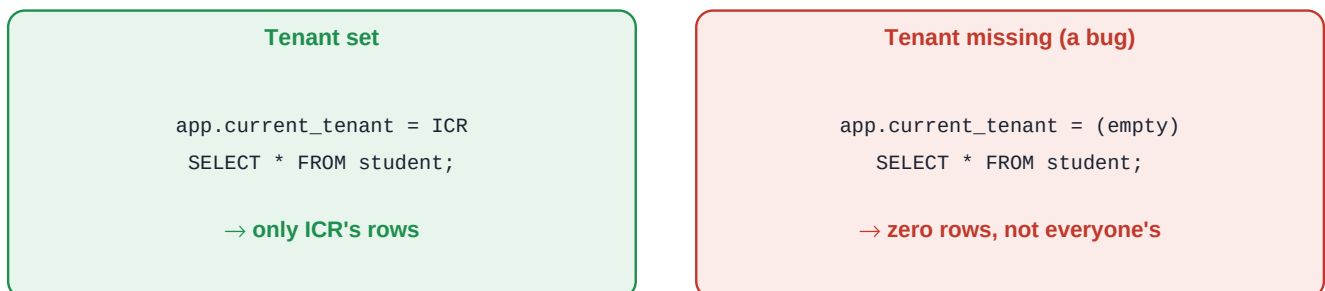
2. Five locks between a request and the data

Each lock works on its own. To leak data, all five would have to fail at once.



What “fail-closed” means

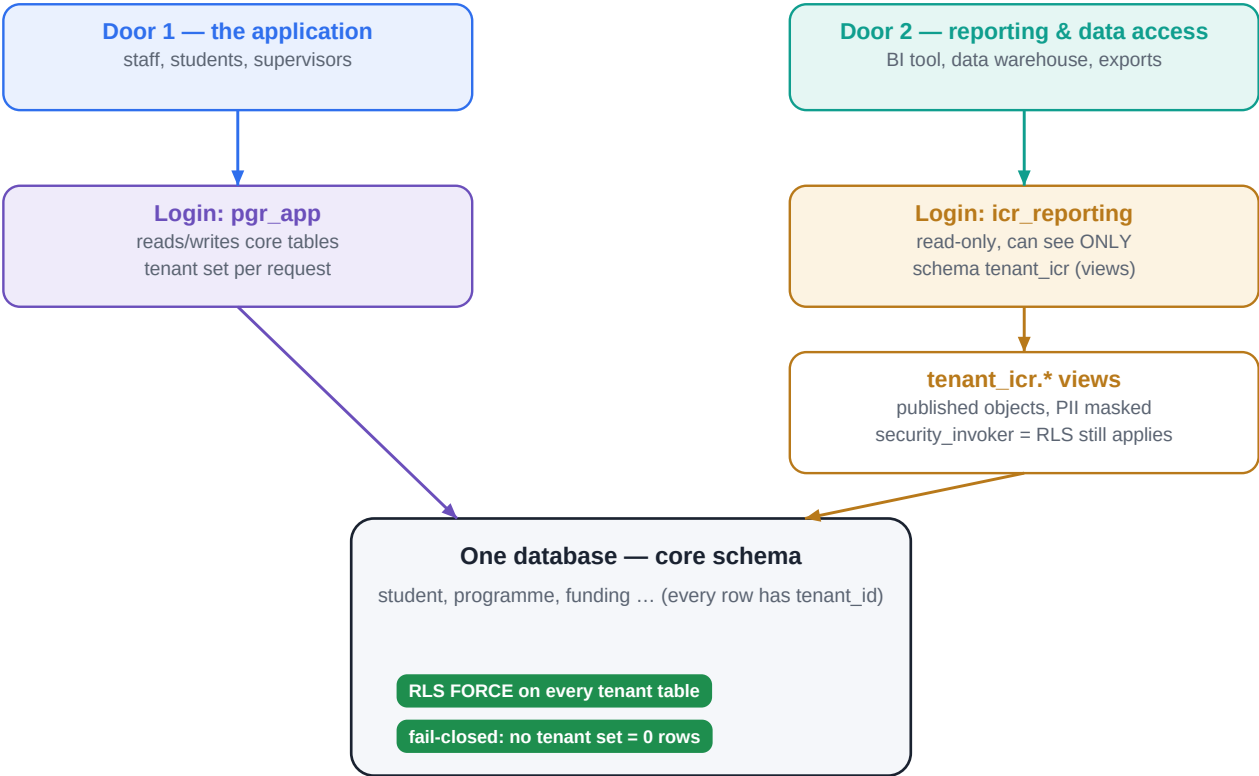
If the tenant is ever missing — a bug, a background job that forgot to set it — the database returns **nothing**, never everything:



3. One database, two doors — where the views fit

This is the senior developer's idea: one main schema, with a layer of views per tenant on top. We use it for the door where it is strongest — **reporting and data access** — while the application keeps going straight to the tables under row-level security.

The owner login (pgr) runs migrations only — never the app, never reporting



	Door 1 — application	Door 2 — reporting / data access
Who	Staff, students, supervisors, background jobs	BI tools, data warehouse, scheduled exports, API consumers
Login	pgr_app (restricted, not the owner)	One read-only login per tenant, e.g. icr_reporting
Sees	Core tables, filtered by row-level security	Only its own schema of views (tenant_icr.*)
Writes	Yes, through the app's business rules	No — read only
Locks	Locks 1–5	Its own schema + views with security_invoker so row-level security still applies (two locks)

Why not send the app through views too?

Writing through views needs `WITH CHECK OPTION` on every view, and the views have to be regenerated for every tenant on every schema change (117 tables × every tenant). Row-level security already does the same filtering inside the database, for reads and writes, with one policy per table. Views add most value where they also **shape and hide** data — which is the reporting door.

4. How it deploys

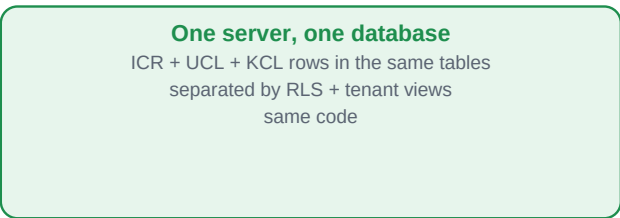
The same code runs both ways, so the choice is per client, not per release.

A. Separate install per institution (silo)



Separation by infrastructure; RLS is a second lock.
Agreed in Demo 2 (item 1.22). Best for large / sensitive clients.

B. Shared install (pool)



Separation by the database rules (the five locks).
Cheaper to run; for smaller clients or a shared tier.

	A. Separate install (silo)	B. Shared install (pool)
Isolation	Separate servers and databases + the five locks	The five locks
Cost to run	Higher (one stack per client)	Lower (one stack)
Best for	ICR and other large or sensitive clients	Smaller clients, trials, a shared tier
Backups / restore	Per client, simple	Per-tenant export / restore tools (built)

5. Outside the database

Leaks also happen around the database. The same rule — every key carries the tenant — applies everywhere data is stored or cached:

Area	Rule
Uploaded files and exports	Stored under a per-tenant prefix (or bucket); links are signed and expire
Caches	Every cache key starts with the tenant id
Background jobs	Each job carries its tenant and sets it on its transaction; no tenant = no rows
Search and AI context	Only this tenant's rows are retrieved; prompts never mix tenants
Logs and errors	Tenant id on every line; no personal data in logs
Support / admin access	Named, time-limited, audited; no shared owner login

6. What is built and what is left

Item	Status
Lock 1 — tenant from subdomain, checked against the token	Built (MT-5)
Lock 2 — tenant set per transaction	Built (incl. the read-replica fix)
Lock 3 — restricted login pgr_app	Partly: built (MT-6) but optional; dev doesn't use it yet
Lock 4 — row-level security	Partly: 116 of 117 tables (missing: composer_run); tenant_id still nullable
Lock 5 — guard tests, two-tenant tests	To build
Tenant view layer + reporting logins	To build (feeds the data warehouse export)
Per-tenant files / caches / jobs review	Partly: files and jobs carry the tenant; full review to do
Independent security test	To do: recommended before production

Suggested order

- 1 Finish the locks: RLS on composer_run, tenant_id NOT NULL, run dev on pgr_app (about 1 day).
- 2 Guard test + two-tenant tests over every endpoint group (about 1–2 days).
- 3 Tenant view layer + per-tenant reporting logins (about 3–4 days) — then the data warehouse export reads only from it.
- 4 Review files, caches, jobs, AI context; then an independent penetration test.

Questions for the senior developer: is the view layer meant for the application or for reporting? One schema of views per tenant, or one set filtered by the session's tenant? Does it need to serve separate installs as well as the shared one?